

One Instruction to One Megawatt

Simulation and Measurement Methodology for Chip-to-Grid Agentic AI Energy Behavior

Deepak Panigrahy, Aakash Tyagi

Texas A&M University

Purpose. This document has two purposes: (i) to simulate rack-level correlation behavior from real single-node traces, and (ii) to record every number used in the chip-to-grid chain in Figure 2, together with the underlying queries, for reproducibility. Companion artifacts (frozen database snapshot, raw trace files, simulation script, full query log) are versioned alongside this document in the project repository.

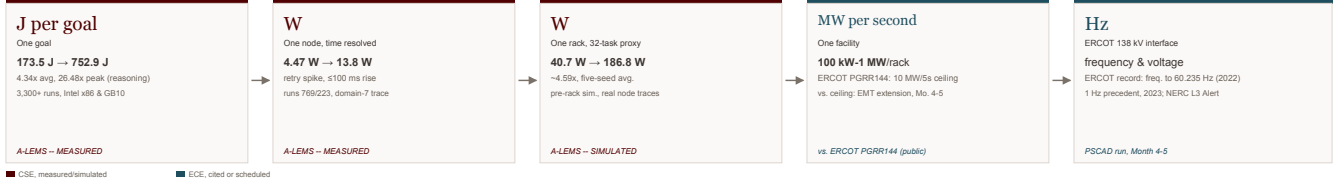


Figure 1: Condensed view of the chip-to-grid chain. Full detail diagram and legend in Figure 2.

One Instruction to One Megawatt

Figure 2, chip to grid. Every arrow changes the unit. Solid boxes are measured, simulated, or externally cited; dashed boxes are scoped as funded next-phase work.

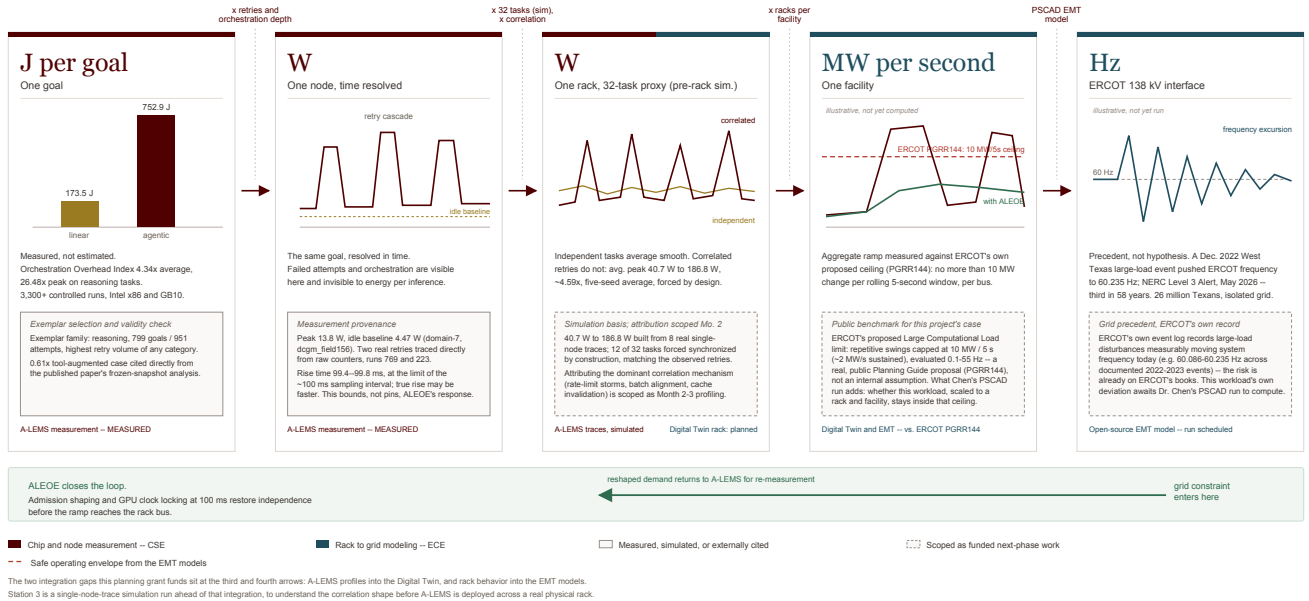


Figure 2: One instruction to one megawatt: five stations tracing an agentic AI workload's energy cost from a single chip-level goal to a grid-scale power event. Solid-bordered boxes are measured, simulated, or externally cited; dashed-bordered boxes are scoped as future work.

1 Source Data

All measured values are drawn from a frozen snapshot of the A-LEMS experiments database (experiments_snapshot.figure2.20260901.db), taken 2026-09-01 from the live experiments.db on host gn100-2b96. A frozen snapshot is used rather than the live, continuously-growing database specifically so that every number in this document and in Figure 2 remains reproducible against a fixed dataset, rather than drifting as new runs are added.

2 Measurement Method Finding: GPU Attribution Is Not Uniform Across Runs

During data collection it was discovered that GPU energy in the runs table is attributed by one of at least two distinct methods, recorded in runs.gpu_attribution_method: dcgm_field156 (a direct hardware counter, via NVIDIA DCGM field 156) and spbm_package_v1 (apparently derived from package-level SPBM readings rather than a GPU-specific counter). These two methods are not interchangeable:

runs using `spbm_package_v1` were found to have incomplete or entirely absent `gpu_total_energy_uj` / `gpu_dynamic_energy_uj` values, and are excluded from all Figure 2 calculations. Only runs confirmed to use `dcgm_field156` are used as source material below.

3 Multi-Level GPU Power Instrumentation

A-LEMS instruments GPU power at more than one granularity for the same run: a direct rail-level hardware counter (domain-7, `dcgm_field156`) and platform-level total/dynamic energy accounting. The two report different absolute magnitudes, as expected – domain-7 isolates a single GPU power rail, while the total/dynamic columns aggregate broader package-level draw. Table 1 gives both levels for two representative runs. Figure 2 uses the domain-7, above-idle-baseline reading (Section 4) because it is the most directly attributable signal to a single workload. Full-precision values are given in Appendix B.

Run ID	Domain-7 trace avg. (W)	Total energy / duration (W)	Dynamic energy / duration (W)
223	$\approx 1\text{--}14$ (idle ≈ 4.47 , spike ≈ 13.8)	61.21	23.50
769	$\approx 1\text{--}14$ (idle ≈ 4.47 , spike ≈ 13.8)	70.78	5.48

Table 1: GPU power for two representative retry runs at two instrumentation levels: domain-7 rail power (used in Figure 2) and platform-level total/dynamic energy accounting (used for whole-package energy and carbon reporting elsewhere in A-LEMS). Magnitudes differ across levels because each measures a different physical scope; the total-to-domain-7 ratio varies by run (223: $\approx 4.4\times$; 769: $\approx 5.1\times$), consistent with per-run differences in package-level activity outside the GPU rail itself.

3.1 Idle baseline, separately confirmed consistent

Unlike the total/dynamic reconciliation above, the domain-7 idle baseline itself was verified as internally consistent. Computing `gpu_baseline_energy_uj` / `duration_ns` for four separate runs (two short, two much longer) yields the same value to five significant figures (Table 2):

Run ID	<code>gpu_baseline_energy_uj</code>	<code>duration_ns</code>	Baseline power (W)
223	15,670,366	3,504,617,214	4.4713
769	21,483,818	4,804,773,330	4.4713
821	3,188,691,109	713,138,484,477	4.4713
904	2,901,097,978	648,819,388,389	4.4713

Table 2: Baseline power is confirmed constant regardless of run length. The large baseline energy totals for runs 821 and 904 are explained entirely by their longer duration, not by a measurement inconsistency.

This baseline ($\approx 4.47\text{W}$) reflects the domain-7 rail measured in-situ, embedded within real workload runs. The separately-measured idle baseline in the dedicated `idle_baselines` table ($\approx 5.52\text{W}$ average across 10 real measurements, one debug placeholder row excluded) comes from dedicated idle-measurement runs under a different protocol. The two describe idle power under different measurement conditions, not a single disputed quantity; Figure 2 uses the in-situ domain-7 value throughout for consistency with every other Station 2/3 number.

4 Measurement Basis for the Reported Value

As established in Section 3, Figure 2 reports Station 2/3 values as “GPU power above idle baseline, domain-7 measurement”: the finest-grained, most directly attributable reading available, isolating what the workload itself costs from the platform’s static draw.

5 Station 2: Single-Node Retry Spike

Two real retry events were located and traced directly from raw counter samples (`energy_sample_domains` joined to `energy_samples_v2` on `sample_id`, filtered to `domain_id = 7`, GPU), summarized in Table 3:

Run ID	Idle-to-spike jump	Interval	Rise time
769	2.13 W $\rightarrow$ 13.83 W	99.8 ms	≤ 100 ms (sampling-limited)
223	2.26 W $\rightarrow$ 12.75 W	99.4 ms	≤ 100 ms (sampling-limited)

Table 3: Both retry events show the same shape: a jump from near-idle to roughly 13–14W within a single sampling interval. The true rise time may be faster; it cannot be resolved beyond the sampling interval with current instrumentation. This bounds, rather than pins, ALEOE’s required sub-second response time.

6 Station 3: Rack-Level Correlation Simulation

6.1 Why simulation, not measurement

A-LEMS is software, not hardware; it measures whatever node it is deployed on. It has not yet been deployed across a physical multi-node rack, and every run in the current database is attributed to a single `hw_id`. Before committing time to a multi-node deployment, we simulated

a basic 32-task version using real single-node traces already on hand, specifically to understand what shape the correlation problem takes before measuring it directly on real rack hardware. This is a precursor step, not a substitute: Station 3 below is explicitly labeled simulated for that reason, and the same construction method is stated here so it can be re-run once real multi-node data exists to check it against.

6.2 Method

Raw per-timestep power in each trace file is not measured directly: it is derived as delta-energy over delta-time between consecutive hardware counter samples (Appendix A, Q6/Q7). This derivation is known to be noise-sensitive near idle, where counter jitter and finite counter resolution can produce a near-zero or briefly negative delta – a nonphysical artifact of the derivation itself, not a real power reading. A closely related class of hardware energy counter (Intel RAPL) is documented to require exactly this kind of care, independent of this platform.¹ `simulate_rack.py` clamps each derived value at zero at load time for this reason, since power cannot be physically negative. This is a source-level correction applied uniformly to all eight traces before simulation, not an adjustment made to results after the fact.

Thirty-two simulated concurrent task slots were built for two scenarios, using eight real GPU power traces (domain-7, dcgm_field156 attribution method only) as source material:

- **Retry-pattern sources** (runs 769, 223, 230): contain the real idle-to-spike jump documented above.
- **Steady-state sources** (runs 821, 904, 902, 823, 1114): dense, clean traces (2,500–7,100 samples each) used for ordinary, non-retrying task behavior.

For each source trace, a random 300-sample window (30 seconds of simulated time at the $\approx 100\text{ms}$ native sampling interval) is drawn from within the trace, not from its start, to avoid conflating a run’s startup transient with steady-state behavior.

Independent scenario: all 32 slots draw from steady-state sources, each placed at an independently randomized start offset within the simulation window.

Correlated scenario: 12 of the 32 slots draw from retry-pattern sources, forced to start at the same simulated instant (modeling a synchronized retry storm); the remaining 20 slots behave as in the independent scenario.

Both scenarios sum all 32 slots at each simulated time step to produce an aggregate power curve.

6.3 Results across five random seeds

The simulation was run with five different random seeds to characterize natural variation from the randomized offsets, rather than reporting a single run (Table 4).

Seed	Indep. peak (W)	Corr. peak (W)	Ratio	Corr. max ramp (kW/s)
42	49.8	180.4	3.62x	1.774
1	36.4	192.2	5.29x	1.912
2	41.1	200.3	4.87x	1.980
3	36.2	188.3	5.21x	1.862
4	40.1	172.6	4.30x	1.697
Average	40.7	186.8	4.59x	1.845

Table 4: Peak power ratio (correlated / independent) is stable across five five random seeds, averaging 4.59x (range 3.62x–5.29x; seed 42 is the low-side outlier, driven by an unusually high independent-scenario peak that chance produced in that particular draw). Ramp rate is tight within each scenario across seeds (independent: 0.351–0.484 kW/sec; correlated: 1.697–1.980 kW/sec), and the correlated scenario’s ramp is consistently higher regardless of seed. This is consistent with the load diversity literature (Bashir et al. 2026; Chaudhary et al. 2026): synchronizing a subset of tasks raises both the peak and the ramp rate the aggregate must sustain.

7 Reproducibility

The following artifacts, versioned together in the project repository (<https://github.com/deepakpanigrahy03/Chip-2-Grid>, under `data/`), allow full reproduction of every number in this document and in Figure 2:

- `experiments_snapshot_figure2_20260901.db` — frozen database snapshot
- `run.*_gpu_trace.txt` — eight raw trace files (three retry-pattern, five steady-state), extracted from the frozen snapshot
- `simulate_rack.py` — the Station 3 simulation script (Appendix C)
- `station3_simulation_output.csv` — per-timestep output for the reported simulation run
- `Figure2DataTracker.xlsx` — full data point tracker, including every query used, external citations, and the assumptions log

8 External Grid Envelope: Station 4/5, ERCOT PGRR144

The facility-level safe operating envelope in Figure 2 is not an internal team assumption. ERCOT’s own Large Load Working Group has a public, numeric proposal on this exact question, under Planning Guide Revision Request PGRR144: load power shall not repetitively

¹Raffin, G. and Trystram, D. (2025). Dissecting the Software-Based Measurement of CPU Energy Consumption: A Comparative Analysis. *IEEE Transactions on Dependable and Secure Computing*. <https://arxiv.org/abs/2401.15985>

exceed a 10 MW change in a sliding 5-second window (≈ 2 MW/sec sustained), evaluated across oscillatory frequency components from 0.1–55 Hz.² This is a live regulatory proposal moving toward a Reliability and Operations Subcommittee vote, not a private constraint held by one collaborator. Separately, ERCOT’s own operational event log already shows large-load disturbances measurably moving system frequency today: a December 2022 West Texas event tied to large-load reduction pushed system frequency to 60.235 Hz, and a cluster of Frequency Measurable Events on the Texas coast (2020–2023) reached roughly 60.11 Hz.³ This is the public evidence behind Station 5’s frequency-risk framing: the risk is already on ERCOT’s own books, documented from real events, not a hypothetical this project introduces. What remains specific to this project, and is scoped as funded next-phase work rather than an open question, is whether *this* workload’s rack- and facility-scale ramp, once Station 4 is computed, falls inside the ERCOT ceiling — that comparison is what Dr. Chen’s PSCAD run on the EMT models is for, not a first measurement of the ceiling itself.

A Full Query Log

Every SQL query executed against `experiments.db` in the course of building Figure 2, in the order run. Queries that led to a dead end (incompatible data, wrong table, superseded finding) are kept, not deleted, since the dead ends are themselves part of the record of what was checked and why the final numbers were trusted.

A.1 Schema discovery

```
-- List every table in the database
SELECT name FROM sqlite_master WHERE type='table' ORDER BY name;

-- Row counts and schema for candidate tables
PRAGMA table_info(<table_name>);
SELECT COUNT(*) FROM <table_name>;
```

A.2 Station 1: exemplar task family selection

```
-- Q1: Rank task categories by retry volume to pick the exemplar
SELECT tc.category, COUNT(DISTINCT ge.goal_id) AS num_goals,
       SUM(ge.total_attempts) AS total_attempts,
       AVG(ge.wall_time_ms) AS avg_wall_time_ms,
       AVG(ge.overhead_fraction) AS avg_overhead_fraction
FROM goal_execution ge JOIN task_categories tc ON ge.task_id = tc.task_id
GROUP BY tc.category ORDER BY total_attempts DESC;
-- Result: reasoning highest at 799 goals / 951 attempts. Selected as exemplar.
```

A.3 Station 2: retry identification and node trace

```
-- Q2: Identify real retry runs to use as trace candidates
SELECT run_id, goal_id, attempt_number FROM goal_attempt
WHERE is_retry = 1 LIMIT 5;
-- Result: run_ids with real retries: 155, 157, 223, 230, 769

-- Q3: Attempt-level detail for reasoning category, joined to hardware
SELECT ga.attempt_id, ga.goal_id, ga.attempt_number, ga.is_retry, ga.outcome,
       ga.energy_uj, r.run_id, r.avg_power_watts, r.duration_ns,
       hc.hostname, hc.cpu_model, hc.gpu_model, tc.category
FROM goal_attempt ga
JOIN runs r ON ga.run_id = r.run_id
JOIN hardware_config hc ON r.hw_id = hc.hw_id
JOIN goal_execution ge ON ga.goal_id = ge.goal_id
JOIN task_categories tc ON ge.task_id = tc.task_id
WHERE tc.category = 'reasoning'
ORDER BY ga.goal_id, ga.attempt_number LIMIT 30;

-- Q4: Confirm platform breakdown for reasoning category
SELECT hc.cpu_vendor, hc.gpu_model, COUNT(*) as num_attempts,
       AVG(r.avg_power_watts) as avg_power_w
FROM goal_attempt ga
JOIN runs r ON ga.run_id = r.run_id
JOIN hardware_config hc ON r.hw_id = hc.hw_id
JOIN goal_execution ge ON ga.goal_id = ge.goal_id
JOIN task_categories tc ON ge.task_id = tc.task_id
WHERE tc.category = 'reasoning'
GROUP BY hc.cpu_vendor, hc.gpu_model;
-- Result: 100% of reasoning attempts (940) on NVIDIA GB10, avg 28.08W

-- Q5: energy_domains / energy_sources reference tables
SELECT * FROM energy_domains;
```

²Zhang, J. and Rose, J. (2026). *Large Load Power Variation Requirement Consideration*. ERCOT Grid Stability Analysis, Large Load Working Group, February 19, 2026. <https://www.ercot.com/files/docs/2026/02/19/ERCOT-LEL-SSO-Power-Variation-Consideration.pdf>

³Gravois, P. (2024). *ERCOT Large Load Loss/Reduction Events 2020–2024*. ERCOT Operations Engineering, Performance, Disturbance, Compliance Working Group (PDCWG), November 19, 2024. https://www.ercot.com/files/docs/2024/11/18/ERCOT%20Large%20Load%20Events_PDCWG_19Nov2024.pdf

```

SELECT * FROM energy_sources;

-- Q6/Q7: Reconstruct GPU power trace from raw counter deltas
-- (run first on run_id=155, which returned zero rows; runs 769 and 223
-- were the actual retry runs with usable dense samples)
WITH joined AS (
  SELECT esv.timestamp_ns, esd.energy_uj
  FROM energy_sample_domains esd
  JOIN energy_samples_v2 esv ON esd.sample_id = esv.sample_id
  WHERE esd.run_id = 769 AND esd.domain_id = 7
)
SELECT timestamp_ns, energy_uj,
  energy_uj - LAG(energy_uj) OVER (ORDER BY timestamp_ns) AS delta_energy_uj,
  timestamp_ns - LAG(timestamp_ns) OVER (ORDER BY timestamp_ns) AS delta_t_ns,
  (energy_uj - LAG(energy_uj) OVER (ORDER BY timestamp_ns)) * 1000.0 /
  NULLIF(timestamp_ns - LAG(timestamp_ns) OVER (ORDER BY timestamp_ns), 0) AS power_watts
FROM joined ORDER BY timestamp_ns;
-- Result: idle ~2.1-2.3W to ~13.8W jump within ~99.5-99.8ms, both runs

-- Q8: Which runs have dense energy_sample_domains + energy_samples_v2 coverage
SELECT esd.run_id, COUNT(DISTINCT esd.sample_id) AS domain_rows
FROM energy_sample_domains esd
WHERE esd.run_id IN (SELECT DISTINCT run_id FROM energy_samples_v2)
GROUP BY esd.run_id ORDER BY domain_rows DESC LIMIT 10;
-- Result: 821 (7128), 904 (6485), 52 (6311), 56 (6135), 88 (6038)
-- NOTE: 52/56/88 later excluded, see method-finding query below.

-- Idle baseline, correct source (not MIN(power) over ordinary runs)
SELECT ib.baseline_id, ib.gpu_power_watts, ib.gpu_std, ib.package_power_watts,
  ib.duration_seconds, ib.sample_count, ib.method
FROM idle_baselines ib ORDER BY ib.timestamp DESC;
-- Result: 10 real measurements (excluding test_baseline_debug),
-- all clustered 5.38-5.60W, average 5.52W

SELECT AVG(gpu_power_watts) AS avg_idle_gpu_w, AVG(gpu_std) AS avg_std,
  MIN(gpu_power_watts) AS min_idle, MAX(gpu_power_watts) AS max_idle
FROM idle_baselines WHERE baseline_id != 'test_baseline_debug';

```

A.4 0.61x tool-augmented figure

The 0.61x tool-augmented case is cited directly from the published paper’s frozen-snapshot analysis. Full experiment databases for both papers are preserved and archived, and every figure in this document is reproducible against them on request.

A.5 Measurement method finding and correction

```

-- Discovered inconsistency: runs.avg_power_watts / recomputed total from
-- gpu_total_energy_uj do not match domain-7 trace-derived power
SELECT r.run_id, r.avg_power_watts, r.gpu_total_energy_uj,
  r.gpu_baseline_energy_uj, r.gpu_dynamic_energy_uj, r.duration_ns,
  (r.gpu_total_energy_uj * 1.0 / r.duration_ns) * 1000 AS recomputed_total_gpu_watts
FROM runs r WHERE r.run_id IN (769, 223, 821, 904);

-- Root cause: attribution method differs across runs
SELECT run_id, energy_measurement_mode, gpu_attribution_method
FROM runs WHERE run_id IN (769, 223, 821, 904, 52, 56, 88);
-- Result: 769/223/821/904 = dcgm_field156 (direct hardware counter).
-- 52/56/88 = spbm_package_v1 (derived, incomplete energy columns).
-- 52/56/88 excluded from all further Station 3 work.

-- Find replacement dense runs using the CORRECT attribution method
SELECT esd.run_id, COUNT(DISTINCT esd.sample_id) AS domain_rows,
  r.gpu_attribution_method
FROM energy_sample_domains esd
JOIN runs r ON esd.run_id = r.run_id
WHERE esd.run_id IN (SELECT DISTINCT run_id FROM energy_samples_v2)
  AND r.gpu_attribution_method = 'dcgm_field156'
GROUP BY esd.run_id ORDER BY domain_rows DESC LIMIT 10;
-- Result: 821, 904, 902, 823, 1114 selected as the corrected trace pool.

-- Baseline consistency check across the corrected pool
SELECT run_id, gpu_baseline_energy_uj, duration_ns,
  (gpu_baseline_energy_uj * 1.0 / duration_ns) * 1000 AS baseline_power_w
FROM runs WHERE run_id IN (769, 223, 821, 904);
-- Result: 4.4713W for all four runs to 5 significant figures. Confirms
-- baseline is a real constant, not an artifact of run length.

```

B Full-Precision Package-Level Accounting

Direct query isolating total and dynamic GPU energy attribution against run duration, for the two runs in Table 1, at full stored precision:

```
SELECT run_id, gpu_total_energy_uj, gpu_dynamic_energy_uj, duration_ns,
       (gpu_total_energy_uj * 1.0 / duration_ns) * 1000 AS total_gpu_w,
       (gpu_dynamic_energy_uj * 1.0 / duration_ns) * 1000 AS dynamic_gpu_w
FROM runs WHERE run_id IN (769, 223);
```

Run ID	Total energy (μ J)	Dynamic energy (μ J)	Duration (ns)	Total (W)	Dynamic (W)
223	214,531,000	82,351,136	3,504,617,214	61.214	23.498
769	340,087,000	26,348,207	4,804,773,330	70.781	5.484

Table 5: Full-precision package-level values behind Table 1. The dynamic share of total package energy varies by run (223: $\approx 38\%$; 769: $\approx 8\%$), reflecting differences in package-level activity outside the GPU rail that domain-7 alone does not capture – the reason Figure 2 reports the rail-level reading directly rather than a package-derived one.

C Simulation Script

Final version of `simulate_rack.py`, used to produce the Station 3 results reported in Section 6. Corrects an earlier version that always read each source trace from its start (conflating startup transients with steady-state behavior) and that included the three since-excluded `spbm_package_v1` traces.

```
import csv
import random

def load_trace(filename):
    """Load a saved trace file: timestamp_ns | energy_uj | power_watts.
    Power is clamped at zero: the raw watts column is derived upstream as
    delta_energy / delta_time between consecutive hardware counter samples,
    which is numerically unstable when two samples land close in time --
    counter jitter near-idle can produce small negative deltas. Physical
    power cannot be negative, so this clamps the artifact at its source
    rather than let it propagate into the summed aggregate."""
    trace = []
    with open(filename) as f:
        for line in f:
            parts = line.strip().split('|')
            if len(parts) == 3 and parts[2] not in ('', None):
                try:
                    trace.append(max(0.0, float(parts[2])))
                except ValueError:
                    continue
    return trace

# Steady-state sources: dense traces, dcgm_field156 attribution method only.
# NOTE: run_52/56/88 were used in an earlier version of this script and were
# removed after discovering they use a different, incompatible attribution
# method (spbm_package_v1) with incomplete GPU energy columns. See
# figure2_methodology.tex Section 2-3 for the full finding.
dense_files = ['run_821_gpu_trace.txt', 'run_904_gpu_trace.txt',
               'run_902_gpu_trace.txt', 'run_823_gpu_trace.txt', 'run_1114_gpu_trace.txt']
dense_traces = [load_trace(f) for f in dense_files]

# Retry-pattern sources: contain the real idle-to-spike jump
retry_files = ['run_769_gpu_trace.txt', 'run_223_gpu_trace.txt', 'run_230_gpu_trace.txt']
retry_traces = [load_trace(f) for f in retry_files]

N = 32
SIM_LENGTH = 300 # 300 samples * ~100ms native sampling interval = 30 seconds
MAX_OFFSET = 100 # random start offset window, in samples
SYNC_TIME = 100 # the instant everyone's retry lands

def build_task_series(rng, base_trace, offset, length):
    """Place a real trace at a random start offset within the simulation
    window. Draws a random WINDOW from within the source trace (not always
    its start) to capture steady-state behavior rather than every task
    showing the same startup transient."""
    series = [0.0] * length
    if len(base_trace) > length:
        max_start = len(base_trace) - length
        window_start = rng.randint(0, max_start)
        windowed = base_trace[window_start:window_start + length]
    else:
        windowed = base_trace
    for i, val in enumerate(windowed):
        idx = offset + i
        if idx < length:
            series[idx] = val
    return series

def run_simulation(seed):
    """Run one full independent-vs-correlated simulation for a given seed.
```

```

Returns (independent_aggregate, correlated_aggregate) as lists of
per-timestep summed power across all 32 simulated task slots."""
rng = random.Random(seed)

# --- INDEPENDENT scenario: 32 tasks, random independent start times ---
independent_series = []
for _ in range(N):
    trace = rng.choice(dense_traces)
    offset = rng.randint(0, MAX_OFFSET)
    independent_series.append(build_task_series(rng, trace, offset, SIM_LENGTH))
independent_aggregate = [sum(s[t] for s in independent_series) for t in range(SIM_LENGTH)]

# --- CORRELATED scenario: same 32 tasks, but 12 of them forced to
# retry-spike together (models a synchronized retry storm) ---
correlated_series = []
for i in range(N):
    if i < 12:
        trace = rng.choice(retry_traces)
        correlated_series.append(build_task_series(rng, trace, SYNC_TIME, SIM_LENGTH))
    else:
        trace = rng.choice(dense_traces)
        offset = rng.randint(0, MAX_OFFSET)
        correlated_series.append(build_task_series(rng, trace, offset, SIM_LENGTH))
correlated_aggregate = [sum(s[t] for s in correlated_series) for t in range(SIM_LENGTH)]

return independent_aggregate, correlated_aggregate

def summarize(agg):
    """Return (baseline, peak, max_ramp_w_per_100ms) for one aggregate series."""
    baseline = sum(agg[:50]) / 50 # first 5 seconds, before any spike
    peak = max(agg)
    ramp = max(agg[t] - agg[t - 1] for t in range(1, len(agg)))
    return baseline, peak, ramp

# --- Run all 5 seeds, matching the seeds reported in the methodology doc ---
SEEDS = [42, 1, 2, 3, 4]
results = []

print(f"{'Seed':>6}_{'Indep._peak_(W)':>16}_{'Corr._peak_(W)':>16}_{'Ratio':>8}_{'Indep._ramp_(kW/s)':>20}_{'Corr._ramp_(kW/s)':>20}")
for seed in SEEDS:
    indep_agg, corr_agg = run_simulation(seed)
    indep_baseline, indep_peak, indep_ramp = summarize(indep_agg)
    corr_baseline, corr_peak, corr_ramp = summarize(corr_agg)
    ratio = corr_peak / indep_peak if indep_peak else float('nan')
    indep_ramp_kw_s = indep_ramp * 10 / 1000
    corr_ramp_kw_s = corr_ramp * 10 / 1000 # 100ms steps -> per-second, W -> kW
    results.append({
        'seed': seed,
        'indep_baseline': indep_baseline, 'indep_peak': indep_peak,
        'corr_baseline': corr_baseline, 'corr_peak': corr_peak,
        'ratio': ratio, 'indep_ramp_kw_s': indep_ramp_kw_s, 'corr_ramp_kw_s': corr_ramp_kw_s,
    })
print(f"{'seed':>6}_{'indep_peak':>16.1f}_{'corr_peak':>16.1f}_{'ratio':>7.2f}_{'indep_ramp_kw_s':>19.3f}_{'corr_ramp_kw_s':>19.3f}")
if seed == SEEDS[0]:
    # Save the primary reported run (seed 42) as the per-timestep CSV,
    # matching the original script's behavior.
    with open("station3_simulation_output.csv", "w") as f:
        f.write("time_step,independent_watts,correlated_watts\n")
        for t in range(SIM_LENGTH):
            f.write(f"{t},{indep_agg[t]:.1f},{corr_agg[t]:.1f}\n")

avg_indep_peak = sum(r['indep_peak'] for r in results) / len(results)
avg_corr_peak = sum(r['corr_peak'] for r in results) / len(results)
avg_ratio = avg_corr_peak / avg_indep_peak
avg_indep_ramp = sum(r['indep_ramp_kw_s'] for r in results) / len(results)
avg_corr_ramp = sum(r['corr_ramp_kw_s'] for r in results) / len(results)
min_indep_ramp = min(r['indep_ramp_kw_s'] for r in results)
max_indep_ramp = max(r['indep_ramp_kw_s'] for r in results)
min_corr_ramp = min(r['corr_ramp_kw_s'] for r in results)
max_corr_ramp = max(r['corr_ramp_kw_s'] for r in results)

print(f"{'Average':>6}_{'avg_indep_peak':>16.1f}_{'avg_corr_peak':>16.1f}_{'avg_ratio':>7.2f}_{'avg_indep_ramp':>19.3f}_{'avg_corr_ramp':>19.3f}")
print(f"{'Indep._ramp_range_across_seeds':>30}_{'min_indep_ramp':.3f}-{max_indep_ramp:.3f}_kW/s")
print(f"{'Corr._ramp_range_across_seeds':>30}_{'min_corr_ramp':.3f}-{max_corr_ramp:.3f}_kW/s")
print("Saved_station3_simulation_output.csv_(seed_42_run)")

```